HTML & HTML5: From Zero to Hero

A theory-first guide with working code for every topic

How to Use This Guide

Every chapter has two parts: **Theory** (what it is and why it exists) and **Code** (a working example you can run). To run any example, copy it into a plain text file, save it with a name ending in `.html` (for example `index.html`), and double-click it to open it in your browser. Any text editor works; VS Code is a popular free choice.

Chapters

1. What Is HTML?
2. Document Structure
3. Text Content
4. Links and Attributes
5. Images and Media
6. Lists
7. Tables
8. Forms
9. Semantic HTML5 Layout
10. Block, Inline, Entities and Comments
11. HTML with CSS and JavaScript
12. Accessibility
13. HTML5 APIs
14. SEO, Metadata and Responsive Basics
15. HTML vs HTML5: The Differences
16. Best Practices and Common Mistakes
17. Capstone: A Complete Page
18. Where to Go Next

Chapter 1: What Is HTML?

Theory

HTML stands for **HyperText Markup Language**.

- **HyperText**: text that contains links to other text, so you can jump between pages.
- **Markup**: content wrapped in labels (tags) that tell the browser what each piece *is* (a heading, a paragraph, a link).
- **Language**: it follows a published standard that every browser implements.

HTML is **not a programming language**. It has no variables, loops or logic. It *describes the structure and meaning* of content.

The three core web languages

Language	Role	Analogy
HTML	Structure and meaning	Skeleton
CSS	Appearance	Skin and clothes
JavaScript	Behaviour and interaction	Muscles

How a browser shows a page

1. You type a URL; the browser requests the file from a server.
2. The server returns HTML as plain text.
3. The browser's parser reads it and builds a tree of objects called the **DOM** (Document Object Model).
4. The browser combines the DOM with CSS and draws the page; JavaScript can then change the DOM.

Anatomy of an element

- An **element** is usually: start tag + content + end tag, such as `<p>Hello</p>`.
- **Attributes** add extra information to the start tag, written as `name="value"`.
- **Void elements** have no content and no end tag (`<br>`, `<img>`, `<hr>`, `<input>`, `<meta>`, `<link>`).
- Elements can be **nested** inside each other; the nesting forms the tree. Close tags in the reverse order you opened them.

- Tag names are case-insensitive, but lowercase is the universal convention.

A short history

- 1991: Tim Berners-Lee publishes the first description of HTML.
- 1995: HTML 2.0. 1997: HTML 3.2 and 4.0. 1999: HTML 4.01.
- Early 2000s: XHTML, a stricter XML-based version.
- 2004 onward: the WHATWG group starts working on what became HTML5.
- 2014: HTML5 becomes an official W3C Recommendation.
- Today: the standard is maintained as the **HTML Living Standard** by the WHATWG, continuously updated and no longer released in numbered versions. People still say "HTML5" to mean modern HTML.

Code

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my <strong>first</strong> web page.</p>
    <img src="cat.jpg" alt="A sleeping cat"> <!-- void element: no end tag --
>
    <a href="https://example.com" target="_blank">Visit Example</a>
  </body>
</html>
```

In the link above: `a` is the element, `href` and `target` are attributes, and `Visit Example` is the content.

Chapter 2: Document Structure

Theory

Every HTML page follows the same skeleton.

- `<!DOCTYPE html>`: tells the browser to use modern standards mode. In HTML5 it is this short; older versions had long, hard-to-remember doctypes.
- `<html>`: the root element that contains everything. Set `lang` so screen readers and translators know the language.
- `<head>`: information *about* the page that is not displayed directly: title, character encoding, links to stylesheets, scripts, metadata.
- `<body>`: everything the visitor actually sees.

Important `<head>` elements:

- `<meta charset="UTF-8">`: the character encoding; it should be first in the head so text like accents and emoji display correctly.
- `<meta name="viewport" ...>`: makes the page scale properly on phones.
- `<title>`: shown in the browser tab and search results.
- `<link>`: connects external resources such as CSS files and icons.
- `<style>` and `<script>`: embedded CSS and JavaScript.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="A page about learning HTML.">
  <title>Document Structure Demo</title>
  <link rel="stylesheet" href="styles.css">
  <link rel="icon" href="favicon.ico">
  <style>
    body { font-family: sans-serif; }
  </style>
  <script src="app.js" defer></script>
</head>
<body>
  <p>Only what is inside the body is shown on the page.</p>
</body>
</html>
```

Chapter 3: Text Content

Theory

HTML gives text **meaning**, not just looks.

- **Headings** `<h1>` to `<h6>` form an outline. Use one `<h1>` per page for the main title and do not skip levels just to get smaller text (use CSS for size).
- **Paragraphs** `<p>` hold blocks of text. Browsers collapse extra spaces and line breaks, so spacing in your source file does not appear on screen.
- `<br>` forces a line break; `<hr>` draws a thematic break (a horizontal rule).
- `<pre>` preserves whitespace and uses a monospace font; `<code>` marks computer code.
- **Emphasis**: `<strong>` means strong importance, `<em>` means stress emphasis. The older `<b>` and `<i>` only style text without meaning, though HTML5 gives them specific uses (keywords, voice or mood).
- Other inline meaning: `<mark>` (highlight), `<small>` (side comments), `<del>` and `<ins>` (removed and added text), `<sub>` and `<sup>` (subscript, superscript), `<abbr>` (abbreviation), `<q>` (short quote), `<cite>` (title of a work), `<time>` (machine-readable date), `<kbd>` (keyboard input).
- `<blockquote>` is a longer quotation.

Code

```
<h1>Learning HTML</h1>
<h2>Why it matters</h2>
<p>HTML is the <strong>foundation</strong> of every website. It is
<em>not</em> a programming language.</p>

<p>First line<br>Second line after a line break.</p>
<hr>

<p>Water is H<sub>2</sub>O and E = mc<sup>2</sup>.</p>
<p>Price: <del>$20</del> <ins>$15</ins>. This is <mark>highlighted</mark>.
</p>
<p><abbr title="HyperText Markup Language">HTML</abbr> was created in 1991.
</p>
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save. Use the <code>&lt;p&gt;
</code> tag for paragraphs.</p>
```

```
<p>Published on <time datetime="2026-10-19">October 19, 2026</time>.</p>

<blockquote cite="https://example.com">
  <p>The web is for everyone.</p>
</blockquote>
<p>As they say, <q>practice makes perfect</q>.</p>

<pre>
Line one
    indented line two
</pre>
```

Chapter 4: Links and Attributes

Theory

Links are what make the web a *web*. The `<a>` (anchor) element creates them with the `href` attribute.

Types of URLs

- **Absolute**: the full address, `https://example.com/page.html`.
- **Relative**: relative to the current file, `about.html` or `../images/logo.png`.
- **Fragment**: jumps to an element with a matching `id` on a page, `#contact`.
- **Special schemes**: `mailto:` opens an email, `tel:` starts a phone call.

Useful link attributes

- `target="_blank"` opens in a new tab. Pair it with `rel="noopener noreferrer"` for safety.
- `download` asks the browser to download the file instead of opening it.
- `title` adds a tooltip.

Global attributes work on every element:

- `id`: a unique name for one element (used for fragments, CSS, JS).
- `class`: a reusable label shared by many elements.
- `style`: inline CSS (better kept in a stylesheet).
- `title`, `lang`, `hidden`, `tabindex`, `contenteditable`, `draggable`.
- `data-*` (HTML5): your own custom data, like `data-user-id="42"`.

Code

```
<p><a href="https://example.com">Absolute link</a></p>
<p><a href="about.html">Relative link</a> | <a href="../index.html">Parent
folder</a></p>
<p><a href="https://example.com" target="_blank" rel="noopener
noreferrer">Open in new tab</a></p>
<p><a href="mailto:hello@example.com">Email me</a> | <a
href="tel:+911234567890">Call me</a></p>
<p><a href="files/guide.pdf" download>Download the guide</a></p>

<p><a href="#contact">Jump to contact section</a></p>
<!-- ...lots of content... -->
<h2 id="contact">Contact</h2>

<p id="intro" class="note" title="Hover me" data-level="beginner"
contenteditable="true">
  This paragraph has several global attributes. You can even edit it in the
browser.
</p>
<p hidden>This paragraph is hidden.</p>
```

Chapter 5: Images and Media

Theory

Images

- `<img>` is a void element. `src` is the file; `alt` is the text description. `alt` is essential: screen readers read it, and it appears if the image fails to load. Use `alt=""` for purely decorative images.
- Always set `width` and `height` so the browser reserves space and the page does not jump while loading.
- `loading="lazy"` (HTML5-era) delays loading off-screen images.
- `srcset` and `sizes` let the browser choose the best file for the screen. `<picture>` with `<source>` lets you serve different formats or crops.
- `<figure>` with `<figcaption>` (HTML5) groups a visual with its caption.

Audio and video (HTML5)

Before HTML5, playing media required plugins like Flash. HTML5 added native `<audio>` and `<video>` elements. Key attributes: `controls`, `autoplay` (usually blocked unless muted), `loop`, `muted`, `poster`, `preload`. Provide several `<source>` formats as fallbacks and `<track>` for captions and subtitles.

Embedding and graphics

- `<iframe>` embeds another web page, such as a map or video player.
- `<svg>` (HTML5) draws scalable vector graphics inline.
- `<canvas>` (HTML5) provides a pixel drawing surface controlled by JavaScript (see Chapter 13).

Code

```
<img src="photo.jpg" alt="A mountain at sunrise" width="600" height="400"
loading="lazy">

<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
  sizes="(max-width: 600px) 100vw, 600px"
  alt="A mountain at sunrise">

<picture>
  <source srcset="photo.webp" type="image/webp">
  <source srcset="photo.avif" type="image/avif">
  <img src="photo.jpg" alt="A mountain at sunrise">
</picture>

<figure>
  <img src="chart.png" alt="Bar chart of monthly visitors">
  <figcaption>Figure 1: Monthly visitors in 2026.</figcaption>
</figure>

<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>

<video controls width="480" poster="thumb.jpg">
  <source src="movie.mp4" type="video/mp4">
```

```
  <source src="movie.webm" type="video/webm">
  <track kind="subtitles" src="movie-en.vtt" srclang="en" label="English"
default>
  Your browser does not support video.
</video>

<iframe src="https://example.com" title="Embedded example page" width="500"
height="300"></iframe>

<svg width="120" height="120" viewBox="0 0 120 120" role="img" aria-label="A
red circle">
  <circle cx="60" cy="60" r="50" fill="crimson" />
</svg>
```

Chapter 6: Lists

Theory

HTML has three kinds of lists.

- **Unordered list** `<ul>`: items where order does not matter; shown with bullets.
- **Ordered list** `<ol>`: items where order matters; numbered. Attributes: `start`, `reversed`, `type`.
- **Description list** `<dl>`: name-value pairs. `<dt>` is the term and `<dd>` is the description.

Each item in `<ul>` and `<ol>` goes in an `<li>`. Lists can be nested by placing a list *inside* an `<li>`. Navigation menus are conventionally built from lists too.

Code

```
<h3>Shopping list</h3>
<ul>
  <li>Milk</li>
  <li>Bread</li>
  <li>Fruit
    <ul>
      <li>Apples</li>
      <li>Bananas</li>
    </ul>
  </li>
</ul>
```

```
<h3>Recipe steps</h3>
<ol>
  <li>Boil water</li>
  <li>Add pasta</li>
  <li>Drain and serve</li>
</ol>

<ol start="5" reversed>
  <li>Counts down from five</li>
  <li>Four</li>
  <li>Three</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>The language that structures web content.</dd>
  <dt>CSS</dt>
  <dd>The language that styles web content.</dd>
</dl>
```

Chapter 7: Tables

Theory

Tables display **tabular data** (rows and columns of related information). Do not use them for page layout; that is what CSS is for.

- `<table>` is the container; `<tr>` is a row; `<td>` is a data cell; `<th>` is a header cell.
- Group rows with `<thead>`, `<tbody>` and `<tfoot>`.
- `<caption>` gives the table a title.
- `colspan` and `rowspan` merge cells across columns or rows.
- `scope="col"` or `scope="row"` on `<th>` helps assistive technology connect headers to data.

Code

```
<table border="1">
  <caption>Student scores</caption>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Maths</th>
      <th scope="col">Science</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Asha</th>
      <td>92</td>
      <td>88</td>
    </tr>
    <tr>
      <th scope="row">Ravi</th>
      <td>79</td>
      <td>85</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="3">Scores are out of 100.</td>
    </tr>
  </tfoot>
</table>
```

Chapter 8: Forms

Theory

Forms collect input from visitors: logins, searches, surveys, checkouts.

- `<form>` wraps the controls. `action` is where the data is sent; `method` is `get` (data in the URL, for searches) or `post` (data in the request body, for logins and changes).
- `<label>` describes a control. Connect it with `for` matching the control's `id`; clicking the label then focuses the control, and screen readers announce it.

- `<input>` is the workhorse; its `type` attribute decides what it is. The `name` attribute is the key under which the value is sent.
- `<textarea>` is multi-line text; `<select>` with `<option>` is a dropdown; `<button>` submits or triggers actions.
- `<fieldset>` with `<legend>` groups related controls (for example a set of radio buttons).

HTML5 added many input types: `email`, `url`, `tel`, `number`, `range`, `date`, `time`, `datetime-local`, `month`, `week`, `color`, `search`. Phones show a suitable keyboard, and browsers give built-in pickers and validation.

HTML5 built-in validation attributes: `required`, `min`, `max`, `minlength`, `maxlength`, `pattern` (a regular expression), `step`. Other HTML5 helpers: `placeholder`, `autofocus`, `autocomplete`, `multiple`, `readonly`, `disabled`.

New HTML5 form elements: `<datalist>` (suggestions while typing), `<output>` (a calculation result), `<progress>` (task progress), `<meter>` (a value within a known range, like disk usage).

Important: browser validation is a convenience. Always validate again on the server, because users can bypass the browser.

Code

```
<form action="/signup" method="post">
  <fieldset>
    <legend>Create an account</legend>

    <label for="name">Full name</label>
    <input type="text" id="name" name="name" placeholder="Asha Roy" required
autofocus>

    <label for="email">Email</label>
    <input type="email" id="email" name="email" required>

    <label for="pw">Password (8+ characters)</label>
    <input type="password" id="pw" name="password" minlength="8" required>

    <label for="age">Age</label>
    <input type="number" id="age" name="age" min="13" max="120">

    <label for="dob">Date of birth</label>
```

```
    <input type="date" id="dob" name="dob">

    <label for="phone">Phone (10 digits)</label>
    <input type="tel" id="phone" name="phone" pattern="[0-9]{10}">

    <label for="color">Favourite colour</label>
    <input type="color" id="color" name="color" value="#336699">

    <label for="volume">Volume</label>
    <input type="range" id="volume" name="volume" min="0" max="10" value="5">
  </fieldset>

  <fieldset>
    <legend>Plan</legend>
    <label><input type="radio" name="plan" value="free" checked> Free</label>
    <label><input type="radio" name="plan" value="pro"> Pro</label>
  </fieldset>

  <label><input type="checkbox" name="newsletter" value="yes"> Send me the
newsletter</label>

  <label for="country">Country</label>
  <select id="country" name="country">
    <option value="">-- choose --</option>
    <option value="in">India</option>
    <option value="us">United States</option>
    <option value="uk">United Kingdom</option>
  </select>

  <label for="browser">Favourite browser</label>
  <input list="browsers" id="browser" name="browser">
  <datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Safari">
  </datalist>

  <label for="bio">About you</label>
  <textarea id="bio" name="bio" rows="4" cols="40" maxlength="300">
</textarea>

  <label for="avatar">Profile picture</label>
  <input type="file" id="avatar" name="avatar" accept="image/*">
```

```
  <input type="hidden" name="source" value="landing-page">

  <button type="submit">Sign up</button>
  <button type="reset">Clear</button>
</form>

<progress value="70" max="100">70%</progress>
<meter value="0.6" min="0" max="1" low="0.3" high="0.8" optimum="1">60%
</meter>
```

Chapter 9: Semantic HTML5 Layout

Theory

Semantic elements describe *what the content is*, not just how it looks. Before HTML5, developers built every page from `<div id="header">`, `<div id="nav">`, `<div id="footer">` and so on. Machines (search engines, screen readers) could not tell what those divs meant. HTML5 introduced real elements for common page parts.

Element	Meaning
`<header>`	Introductory content for a page or section (logo, title, nav)
`<nav>`	A block of major navigation links
`<main>`	The unique main content of the page (use once)
`<section>`	A thematic group of content, normally with a heading
`<article>`	A self-contained piece that makes sense alone (blog post, news story, comment)
`<aside>`	Related side content (sidebar, pull quote, ad)
`<footer>`	Footer for a page or section (copyright, links)
`<figure>` / `<figcaption>`	A visual with a caption
`<address>`	Contact information for the author

Element	Meaning
`<time>`	A date or time
`<details>` / `<summary>`	A show/hide widget with no JavaScript
`<dialog>`	A modal or non-modal dialog box

Why it matters: better accessibility (screen readers can jump straight to `<main>` or `<nav>`), better SEO, cleaner and more readable code, and easier maintenance. Use `<div>` only when no semantic element fits.

Code

```
<body>
  <header>
    <h1>My Travel Blog</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/posts">Posts</a></li>
        <li><a href="/about">About</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h2>Three Days in Darjeeling</h2>
        <p>By Asha, <time datetime="2026-10-01">1 October 2026</time></p>
      </header>
      <section>
        <h3>Day 1</h3>
        <p>We arrived early and walked through the tea gardens.</p>
      </section>
      <section>
        <h3>Day 2</h3>
        <p>Sunrise at Tiger Hill was worth the cold.</p>
      </section>
      <footer><p>Filed under: Travel</p></footer>
    </article>
```

```
    <aside>
      <h2>Popular posts</h2>
      <ul><li><a href="#">Kolkata street food</a></li></ul>
    </aside>
  </main>

  <footer>
    <address>Contact: <a
href="mailto:hello@example.com">hello@example.com</a></address>
    <p>&copy; 2026 My Travel Blog</p>
  </footer>
</body>
```

Interactive elements with no JavaScript:

```
<details>
  <summary>What is semantic HTML?</summary>
  <p>HTML that describes the meaning of content, not only its appearance.</p>
</details>

<dialog id="hello">
  <p>Hello from a dialog!</p>
  <form method="dialog"><button>Close</button></form>
</dialog>
<button onclick="document.getElementById('hello').showModal()">Open
dialog</button>
```

Chapter 10: Block, Inline, Entities and Comments

Theory

Block vs inline elements

- **Block-level** elements start on a new line and take the full available width: `<div>`, `<p>`, `<h1>` - `<h6>`, `<ul>`, `<section>`, `<table>`, `<form>`.
- **Inline** elements sit inside a line of text and take only the space they need: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`, `<code>`.
- `<div>` is a generic block container and `<span>` is a generic inline container. They carry no meaning; use them as a last resort, mostly to hook CSS or JavaScript.

- In modern HTML5, these categories are formally replaced by *content categories* (flow, phrasing, and so on), but "block" and "inline" remain the practical way to think about default display. CSS can change either with the `display` property.
- A `<p>` cannot contain another `<p>` or a `<div>`; the browser will quietly close the first paragraph.

Character entities

Some characters have special meaning in HTML (`<`, `>`, `&`). To show them literally, write an **entity**: `<`, `>`, `&`, `"`. Others: ` ` (non-breaking space), `©`, `®`, `€`, `♥`. You can also use a numeric form like `©`. With UTF-8 you can often type symbols directly.

Comments

`<!-- ... -->` notes are ignored by the browser. They are still visible to anyone who views the page source, so never put secrets in them.

Code

```
<!-- This is a comment; it is not shown on the page -->
<div>A block element on its own line</div>
<div>Another block element</div>
<p>Inline elements stay <span style="color: crimson;">inside</span> the same
<a href="#">line</a>.</p>

<p>To write a tag as text, use entities: &lt;p&gt;Hello&lt;/p&gt;</p>
<p>Tom &amp; Jerry &mdash; &copy; 2026 &hearts; &euro;50</p>
<p>Two words that never break across lines.</p>
```

Chapter 11: HTML with CSS and JavaScript

Theory

HTML alone is plain. It is combined with the other two languages in three ways each.

CSS

1. **Inline**: the `style` attribute on one element. Quick, but hard to maintain.
2. **Internal**: a `<style>` element in the head. Fine for a single page.

3. **External**: a `.css` file linked with `<link rel="stylesheet">`. Best for real sites because many pages share it and browsers cache it.

JavaScript

1. **Inline event attribute**: `onclick="..."`. Fine for demos, discouraged in large projects.
2. **Internal**: a `<script>` element containing code.
3. **External**: `<script src="app.js"></script>`.

Script placement matters, because a script that runs before the HTML exists cannot find the elements. Solutions: put the script at the end of `<body>`, or use `defer` in the head. `defer` downloads the script while HTML parses and runs it afterwards, in order. `async` runs it as soon as it downloads, in any order (suitable for independent scripts such as analytics). Use `<noscript>` for content shown when JavaScript is off.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML + CSS + JS</title>
  <style>
    .card { border: 1px solid #ccc; border-radius: 8px; padding: 16px; max-
width: 320px; }
    .card h2 { color: teal; }
    button { padding: 8px 14px; cursor: pointer; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Click counter</h2>
    <p>Clicks: <span id="count">0</span></p>
    <button id="btn">Click me</button>
    <p style="color: gray;">(This paragraph uses an inline style.)</p>
  </div>

  <noscript>Please enable JavaScript to use the counter.</noscript>

  <script>
    let clicks = 0;
    const countEl = document.getElementById('count');
```

```
    document.getElementById('btn').addEventListener('click', () => {
      clicks++;
      countEl.textContent = clicks;
    });
  </script>
</body>
</html>
```

Chapter 12: Accessibility

Theory

Accessibility (often abbreviated a11y) means people with disabilities can use your page: blind users with screen readers, keyboard-only users, people with low vision or colour blindness, people with cognitive or motor impairments. It also helps everyone (think of a noisy room, or a broken mouse).

The best accessibility is simply **correct semantic HTML**:

- Use real headings in order, real lists, real buttons and links. A `<button>` is keyboard-focusable and works with Enter and Space for free; a clickable `<div>` does not.
- Give every image meaningful `alt` text.
- Connect every form control to a `<label>`.
- Use landmarks (`<header>`, `<nav>`, `<main>`, `<footer>`) so users can jump around.
- Set `lang` on `<html>`.
- Ensure enough colour contrast and never rely on colour alone to convey meaning.

ARIA (Accessible Rich Internet Applications) attributes add extra information when HTML alone is not enough: `role`, `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-expanded`, `aria-hidden`, `aria-live`. The first rule of ARIA: *if a native HTML element does the job, use it instead*. Bad ARIA is worse than none.

Other helpers: `tabindex="0"` (make focusable in order), a "skip to main content" link, and `<track>` captions on video.

Code

```
<a href="#main" class="skip">Skip to main content</a>

<nav aria-label="Primary">
  <a href="/">Home</a>
  <a href="/contact">Contact</a>
</nav>

<main id="main">
  <h1>Accessible page</h1>

  <img src="team.jpg" alt="Five colleagues laughing around a table">
  <img src="divider.png" alt=""> <!-- decorative: empty alt -->

  <button type="button" aria-label="Close menu">&times;</button>

  <button type="button" aria-expanded="false" aria-controls="panel">Show
details</button>
  <div id="panel" hidden>Extra details here.</div>

  <label for="user">Username</label>
  <input id="user" name="user" aria-describedby="user-hint">
  <p id="user-hint">Use 4 to 12 letters or numbers.</p>

  <p role="status" aria-live="polite">Form saved.</p>
</main>
```

Chapter 13: HTML5 APIs

Theory

HTML5 is more than new tags. It also standardised a set of **JavaScript APIs** that let web pages do things that used to need plugins or native apps. These are accessed through JavaScript, but they are part of the HTML5 family.

- **Canvas**: a bitmap surface you draw on with JavaScript (charts, games, image effects). Pixels, not elements: once drawn, shapes are not individually editable. Compare with **SVG**, which keeps shapes as DOM elements and scales without blur.

- **Web Storage** (`localStorage` , `sessionStorage`): store small key-value strings in the browser. `localStorage` persists until cleared; `sessionStorage` lasts for the tab session. This replaced most uses of cookies for client-side data. Typical limit is about 5 MB; never store passwords or secrets there.
- **Geolocation**: ask the user (with permission) for their position.
- **Drag and Drop**: make elements draggable with the `draggable` attribute plus drag events.
- **Web Workers**: run JavaScript in a background thread so the page stays responsive.
- **History API**: change the URL and navigation history without reloading (the basis of single-page apps).
- **Others worth knowing**: IndexedDB (larger structured storage), Web Sockets (two-way live connections), Fullscreen, Notifications, Service Workers (offline support), and the Media APIs (camera and microphone).

Code

Canvas

```
<canvas id="board" width="300" height="150" style="border:1px solid #999">
</canvas>
<script>
  const ctx = document.getElementById('board').getContext('2d');
  ctx.fillStyle = 'teal';
  ctx.fillRect(20, 20, 100, 60);          // rectangle
  ctx.beginPath();
  ctx.arc(200, 75, 40, 0, Math.PI * 2);   // circle
  ctx.fillStyle = 'orange';
  ctx.fill();
  ctx.fillStyle = 'black';
  ctx.font = '16px sans-serif';
  ctx.fillText('Hello Canvas', 20, 120);
</script>
```

Web Storage

```
<input id="note" placeholder="Type a note">
<button id="save">Save</button>
<p id="out"></p>
<script>
  const note = document.getElementById('note');
  const out = document.getElementById('out');
  out.textContent = localStorage.getItem('note') || 'Nothing saved yet.';
  document.getElementById('save').addEventListener('click', () => {
    localStorage.setItem('note', note.value);
    out.textContent = 'Saved: ' + note.value;
  });
</script>
```

Geolocation

```
<button id="where">Where am I?</button>
<p id="pos"></p>
<script>
  document.getElementById('where').addEventListener('click', () => {
    navigator.geolocation.getCurrentPosition(
      p => pos.textContent = `Lat ${p.coords.latitude}, Lon
${p.coords.longitude}`,
      err => pos.textContent = 'Could not get location: ' + err.message
    );
  });
</script>
```

Drag and Drop

```
<style>
  #box { width: 80px; height: 80px; background: tomato; }
  #zone { width: 200px; height: 120px; border: 2px dashed gray; margin-top:
10px; }
</style>
<div id="box" draggable="true">Drag me</div>
<div id="zone">Drop here</div>
<script>
  const box = document.getElementById('box');
  const zone = document.getElementById('zone');
  box.addEventListener('dragstart', e => e.dataTransfer.setData('text/plain',
'box'));
  zone.addEventListener('dragover', e => e.preventDefault());   // allow
dropping
  zone.addEventListener('drop', e => { e.preventDefault();
zone.appendChild(box); });
</script>
```

Web Worker (the worker code goes in its own file, `worker.js`)

```
// worker.js
onmessage = e => {
  let sum = 0;
  for (let i = 0; i < e.data; i++) sum += i;
  postMessage(sum);
};
```

```
<script>
  const worker = new Worker('worker.js');
  worker.onmessage = e => console.log('Sum is', e.data);
  worker.postMessage(100000000);
  console.log('The page stays responsive meanwhile.');
</script>
```

Chapter 14: SEO, Metadata and Responsive Basics

Theory

Metadata lives in the `<head>` and helps browsers, search engines and social networks understand your page.

- `<title>` and `<meta name="description">` are what search results typically display. Keep titles specific and descriptions to about one or two sentences.
- `<meta name="viewport" content="width=device-width, initial-scale=1">` is required for mobile-friendly pages.
- **Open Graph** tags (`og:title` , `og:description` , `og:image`) control the preview card when a link is shared on social platforms.
- `<link rel="canonical">` tells search engines the preferred URL when similar pages exist.
- `<meta name="robots">` can ask search engines not to index a page.
- Semantic structure (headings, `<main>` , `<article>`), descriptive link text and meaningful `alt` text all help SEO as well as accessibility.

Responsive design basics: the viewport tag plus CSS media queries. On the HTML side, flexible images (`srcset` , `<picture>`) and relative units help. Link text like "click here" is poor for both SEO and accessibility; describe the destination instead.

Code

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Learn HTML5 Step by Step | My Site</title>
  <meta name="description" content="A beginner-friendly guide to HTML and
HTML5 with examples.">
  <link rel="canonical" href="https://example.com/learn-html">
  <meta name="robots" content="index, follow">

  <!-- Open Graph for social sharing -->
  <meta property="og:title" content="Learn HTML5 Step by Step">
  <meta property="og:description" content="A beginner-friendly guide with
examples.">
  <meta property="og:image" content="https://example.com/preview.jpg">
  <meta property="og:type" content="article">

  <link rel="icon" href="/favicon.ico">
  <meta name="theme-color" content="#0f766e">
</head>
```

A tiny responsive example:

```
<style>
  .grid { display: grid; gap: 12px; grid-template-columns: 1fr; }
  @media (min-width: 700px) {
    .grid { grid-template-columns: repeat(3, 1fr); }
  }
  img { max-width: 100%; height: auto; }
</style>
<div class="grid">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

Chapter 15: HTML vs HTML5: The Differences

Theory

"HTML" in this question usually means **HTML 4.01 / XHTML** (the standard of the late 1990s and 2000s), and "HTML5" means the modern standard. HTML5 was designed to add features for today's web **without breaking old pages**, and to define exactly how browsers should handle even malformed markup, so that all browsers behave the same.

Area	HTML 4.01 / XHTML	HTML5
Doctype	Long, hard to remember, referenced a DTD	Simple: `<!DOCTYPE html>`
Character encoding	Long `<meta http-equiv=...>` form	`<meta charset="UTF-8">`
Page structure	`<div id="header">`, `<div id="footer">` etc.	Semantic: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`
Audio and video	Needed plugins (Flash, Silverlight, RealPlayer)	Native `<audio>` and `<video>` with `<track>` captions
Graphics	Images or plugins	`<canvas>` and inline `<svg>`

Area	HTML 4.01 / XHTML	HTML5
Forms	Few input types (text, password, checkbox, radio, etc.), validation by JavaScript	New types (`email` , `date` , `number` , `range` , `color` ...), built-in validation, `placeholder` , `required` , `<datalist>` , `<output>`
Storage	Cookies only (small, sent with every request)	`localStorage` , `sessionStorage` , IndexedDB
APIs	Little standardisation	Geolocation, Drag and Drop, Web Workers, History, WebSockets, and more
Parsing	Loosely specified; browsers differed	Precisely specified parsing algorithm; consistent error handling
Syntax	HTML4 was SGML-based; XHTML required strict XML (lowercase, closed tags, quoted attributes)	Relaxed: HTML or XML-style syntax both allowed (` ` or ` `)
Custom data	Misusing attributes or classes	`data-*` attributes
Presentational tags	`<font>` , `<center>` , `<big>` , `<strike>` , `<frame>` , `<acronym>` were common	These are **removed**; use CSS (and `<abbr>` , `<del>`)
New text elements	N/A	`<mark>` , `<time>` , `<figure>` , `<figcaption>` , `<details>` , `<summary>` , `<dialog>` , `<progress>` , `<meter>`
Mobile	Not a design concern	Designed with mobile and responsive use in mind
Development model	Numbered versions (3.2, 4.0, 4.01)	Living Standard, continuously updated

What HTML5 removed or deprecated: `<font>` , `<center>` , `<big>` , `<tt>` , `<strike>` , `<basefont>` , `<frame>` , `<frameset>` , `<noframes>` , `<acronym>` , `<applet>` , `<dir>` , and presentational attributes like `align` , `bgcolor` , `border` on most elements. Appearance is now CSS's job.

What stayed the same: the core idea of tags, elements, attributes, nesting and links; the basic elements for headings, paragraphs, lists, tables and forms.

A note on terminology: today's HTML *is* HTML5 and beyond. Saying "HTML5" is shorthand for "modern HTML with its APIs"; there is no separate HTML6 to learn first.

Code

The same simple page written in the old style and the modern style.

HTML 4.01 style

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
  "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <title>Old Page</title>
</head>
<body bgcolor="#ffffff">
  <div id="header">
    <center><font color="blue" size="5">My Site</font></center>
  </div>
  <div id="nav">
    <a href="index.html">Home</a>
  </div>
  <div id="content">
    <p>Welcome!</p>
    <object data="movie.swf" type="application/x-shockwave-flash"></object>
  </div>
  <div id="footer">Copyright 2005</div>
</body>
</html>
```

HTML5 style

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Modern Page</title>
  <style>
```

```
    header h1 { color: blue; text-align: center; }
  </style>
</head>
<body>
  <header><h1>My Site</h1></header>
  <nav><a href="index.html">Home</a></nav>
  <main>
    <p>Welcome!</p>
    <video src="movie.mp4" controls width="400"></video>
  </main>
  <footer>&copy; 2026</footer>
</body>
</html>
```

Chapter 16: Best Practices and Common Mistakes

Theory

Best practices

- Start every page with `<!DOCTYPE html>`, `lang` and `<meta charset="UTF-8">`.
- Choose the element for its **meaning**, then style it with CSS.
- Use lowercase tag and attribute names, always quote attribute values, and indent nested elements consistently.
- Close every element that needs closing; keep nesting correct.
- One `<h1>` per page; keep heading levels in order.
- Always write `alt` text and associate every form control with a `<label>`.
- Keep CSS and JavaScript in external files for real projects.
- Use meaningful, lowercase file names with hyphens: `about-us.html`.
- Validate your HTML with the W3C Markup Validator (validator.w3.org) and test in more than one browser and on a phone.

Common beginner mistakes

- Using `<br>` repeatedly or ` ` to create spacing (use CSS margins).
- Using tables for page layout.
- Skipping heading levels to get a smaller font.
- Using `<div>` and `<span>` for everything.

- Using `<b>` and `<i>` only for looks when `<strong>` and `<em>` fit.
- Duplicate `id` values (an `id` must be unique on a page).
- Forgetting `rel="noopener noreferrer"` with `target="_blank"`.
- Relying only on browser validation for forms.
- Wrapping block elements in invalid parents (for example a `<div>` inside a `<p>`).

Code

```
<!-- Bad -->
<div onclick="go()">Click</div>
<img src="a.jpg">
<font color=red>Warning</font>
<br><br><br>
<p>Text<p>More text

<!-- Good -->
<button type="button" id="go-btn">Click</button>
<img src="a.jpg" alt="Description of the picture" width="300" height="200">
<p class="warning">Warning</p>
<p>Text</p>
<p>More text</p>
```

Chapter 17: Capstone: A Complete Page

Theory

This final example combines almost everything you have learned: document structure, metadata, semantic layout, headings, text, links, images, lists, a table, a form with validation, a `<details>` widget, a canvas, CSS and JavaScript, plus accessibility features. Read it top to bottom and match each part to its chapter.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="A complete HTML5 demo page for
```

```
learners.">
  <title>HTML5 Capstone Page</title>
  <style>
    body { font-family: system-ui, sans-serif; margin: 0; line-height: 1.6;
color: #1f2937; }
    header, footer { background: #0f766e; color: white; padding: 16px 24px; }
    nav a { color: white; margin-right: 16px; }
    main { max-width: 800px; margin: 0 auto; padding: 24px; }
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #d1d5db; padding: 8px; text-align: left; }
    label { display: block; margin-top: 10px; }
    .skip { position: absolute; left: -999px; }
    .skip:focus { left: 8px; top: 8px; background: white; color: black;
padding: 4px; }
  </style>
</head>
<body>
  <a class="skip" href="#main">Skip to main content</a>

  <header>
    <h1>Learn HTML5</h1>
    <nav aria-label="Primary">
      <a href="#basics">Basics</a>
      <a href="#table">Table</a>
      <a href="#form">Form</a>
      <a href="#canvas">Canvas</a>
    </nav>
  </header>

  <main id="main">
    <section id="basics">
      <h2>The Basics</h2>
      <p>HTML is the <strong>structure</strong> of the web. Modern HTML is
often called <em>HTML5</em>.</p>
      <figure>
        <img src="html5.png" alt="The HTML5 logo" width="120" height="120">
        <figcaption>The HTML5 logo.</figcaption>
      </figure>
      <h3>Topics covered</h3>
      <ol>
        <li>Structure</li>
        <li>Text and links</li>
        <li>Forms and tables</li>
```

```
      </ol>
      <details>
        <summary>Is HTML a programming language?</summary>
        <p>No. It describes structure; JavaScript adds logic.</p>
      </details>
    </section>

    <section id="table">
      <h2>Element Comparison</h2>
      <table>
        <caption>Old and new equivalents</caption>
        <thead>
          <tr><th scope="col">Old approach</th><th scope="col">HTML5
element</th></tr>
        </thead>
        <tbody>
          <tr><td>&lt;div id="header"&gt;</td><td>&lt;header&gt;</td></tr>
          <tr><td>&lt;div id="nav"&gt;</td><td>&lt;nav&gt;</td></tr>
          <tr><td>Flash video</td><td>&lt;video&gt;</td></tr>
        </tbody>
      </table>
    </section>

    <section id="form">
      <h2>Contact Us</h2>
      <form action="/contact" method="post">
        <label for="n">Name</label>
        <input id="n" name="name" required>
        <label for="e">Email</label>
        <input id="e" type="email" name="email" required>
        <label for="m">Message</label>
        <textarea id="m" name="message" rows="4" required></textarea>
        <p><button type="submit">Send</button></p>
      </form>
    </section>

    <section id="canvas">
      <h2>Canvas Drawing</h2>
      <canvas id="c" width="300" height="120" aria-label="A teal rectangle
and an orange circle" role="img"></canvas>
      <p><button id="theme" type="button">Toggle dark mode</button></p>
    </section>
  </main>
```

```
  <footer>
    <p>&copy; 2026 Learn HTML5. <a href="mailto:hello@example.com"
style="color:white">Email us</a></p>
  </footer>

  <script>
    const ctx = document.getElementById('c').getContext('2d');
    ctx.fillStyle = 'teal';   ctx.fillRect(10, 10, 120, 80);
    ctx.fillStyle = 'orange'; ctx.beginPath(); ctx.arc(200, 60, 40, 0,
Math.PI * 2); ctx.fill();

    document.getElementById('theme').addEventListener('click', () => {
      document.body.style.background = document.body.style.background ? '' :
'#111827';
      document.body.style.color = document.body.style.color ? '' : '#f9fafb';
    });
  </script>
</body>
</html>
```

Chapter 18: Where to Go Next

Theory

HTML is only the first layer. A sensible learning path after this guide:

1. **Practice**: rebuild a page you like (a blog, a menu, a profile page) from scratch.
2. **CSS**: selectors, the box model, Flexbox, Grid, responsive design, variables and animations.
3. **JavaScript**: variables, functions, the DOM, events, `fetch` for loading data.
4. **Developer tools**: press F12 in your browser to inspect elements, edit HTML live and debug.
5. **Version control**: learn Git and publish a site on GitHub Pages or similar hosting.
6. **Accessibility and performance**: Lighthouse audits, keyboard testing, optimised images.
7. **Frameworks** (later): React, Vue or similar, only after the basics feel natural.

Reference sources: MDN Web Docs (developer.mozilla.org) for authoritative documentation, the HTML Living Standard (html.spec.whatwg.org) for the official specification, and the W3C validator for checking your work.

Practice exercises

1. Build a personal profile page with a heading, photo, bio, a list of hobbies and links to your favourite sites.
2. Build a recipe page using `<article>`, `<figure>`, an ordered list for steps and a table for nutrition.
3. Build a registration form using at least six different input types and built-in validation.
4. Convert an old-style `<div id="header">` layout into semantic HTML5 elements.
5. Add a canvas drawing and a `localStorage` feature that remembers a visitor's name.